

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC

Trương Quốc Anh - 22001543

Cải Tiến Pruning Hybrid Thích Ứng Để Tìm Vé
Số May Mắn Trong Mạng Nơ-ron Sâu

Khoá luận tốt nghiệp
Ngành: Khoa học máy tính và thông tin
(Chương trình đào tạo chuẩn)

Hà Nội - 2026

Lời cam đoan

Em, sinh viên thực hiện đề tài tiểu luận với tiêu đề “*Cải Tiến Pruning Hybrid Thích Ứng Để Tìm Vế Số May Mắn Trong Mạng Nơ-ron Sâu*”, xin cam đoan rằng toàn bộ nội dung trình bày trong báo cáo là kết quả của quá trình nghiên cứu, học tập và tổng hợp kiến thức của nhóm dưới sự hướng dẫn của giảng viên phụ trách. Trong quá trình thực hiện, em có tham khảo một số tài liệu, công trình khoa học và nguồn thông tin từ Internet nhằm phục vụ cho việc tìm hiểu và xây dựng nội dung. Tất cả các tài liệu được sử dụng đều đã được trích dẫn đầy đủ và đúng theo quy định. Em cam kết rằng báo cáo được thực hiện một cách nghiêm túc, không sao chép nội dung từ bất kỳ công trình nào nếu không có trích dẫn phù hợp. Các số liệu, hình ảnh, kết quả và phân tích được trình bày đều phản ánh trung thực quá trình tìm hiểu và thực nghiệm. Báo cáo không vi phạm các quy định về đạo đức nghiên cứu, bản quyền cũng như pháp luật hiện hành. Em hoàn toàn chịu trách nhiệm về nội dung báo cáo trước giảng viên hướng dẫn và nhà trường.

Lời cảm ơn

Để hoàn thành báo cáo cuối kỳ này, em xin bày tỏ lòng biết ơn sâu sắc đến những người đã tận tình hướng dẫn và hỗ trợ chúng em trong suốt quá trình học tập và nghiên cứu. Trước hết, chúng em xin gửi lời cảm ơn chân thành nhất đến TS. Nguyễn Hải Vinh. Sự hướng dẫn, chỉ bảo tận tình và những góp ý chuyên môn sâu sắc của thầy không chỉ giúp chúng em định hướng đúng đắn trong quá trình thực hiện báo cáo mà còn khơi gợi niềm say mê, hứng thú với lĩnh vực thiết kế và đánh giá thuật toán. Thầy đã kiên nhẫn giải đáp những thắc mắc, đồng thời chia sẻ những kiến thức và kinh nghiệm quý báu, giúp chúng em vượt qua những khó khăn và thách thức gặp phải. Sự hỗ trợ nhiệt tình và kịp thời của các thầy cô trong việc giải đáp các vấn đề kỹ thuật cũng như cung cấp tài liệu tham khảo đã góp phần không nhỏ vào việc hoàn thiện báo cáo này. Cuối cùng, nhóm chúng em xin cảm ơn Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Hà Nội đã tạo điều kiện và môi trường học tập tốt nhất để chúng em có thể tiếp thu kiến thức và phát triển bản thân. Mặc dù đã rất cố gắng, song do kiến thức và thời gian có hạn, báo cáo chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy cô và các bạn để báo cáo được hoàn thiện hơn. Xin chân thành cảm ơn!

Tóm tắt nội dung

Nội dung chính của tiểu luận ...

Contents

1	Tổng quan	1
1.1	Động lực	1
1.2	Vấn đề nghiên cứu	1
1.3	Mục tiêu nghiên cứu	1
1.4	Đóng góp	1
2	Nền tảng & Tổng quan	2
2.1	Giả thuyết vé số may mắn	2
2.2	Phân loại thuật toán	2
2.2.1	Nhóm cắt tỉa sau kết nối	2
2.2.2	Nhóm cắt tỉa giai đoạn sớm	3
2.2.3	Nhóm cắt tỉa tại thời điểm khởi tạo	3
2.2.4	Nhóm cắt tỉa hỗn hợp	3
3	Phương pháp	5
3.1	Thuật toán baseline	5
3.1.1	Iterative Magnitude Pruning (IMP)	5
3.1.2	Early-Bird (EB)	7
3.1.3	Gradient Signal Preservation (GraSP)	10
3.1.4	Synflow	13
3.1.5	Hybrid	17
3.2	Hybrid Improved	19
3.2.1	Điểm yếu của thuật toán Hybrid	19
3.2.2	Đề xuất cải tiến	22
4	Thiết lập thử nghiệm	26
4.1	Bộ dữ liệu	26
4.1.1	MNIST	26

4.1.2	CIFAR-10	26
4.1.3	CIFAR-100	27
4.2	Kiến trúc mạng	27
4.2.1	ResNet-20	28
4.2.2	ResNet-50	28
4.3	Siêu tham số	28
4.4	Chỉ số đánh giá	28
4.4.1	Độ Chính Xác Phân Loại	30
4.4.2	Chỉ Số Nén Mô Hình	30
5	Kết quả	32
6	Thảo luận	33
7	Kết luận	34

List of Figures

Chapter 1

Tổng quan

1.1 Động lực

1.2 Vấn đề nghiên cứu

1.3 Mục tiêu nghiên cứu

1.4 Đóng góp

Chapter 2

Nền tảng & Tổng quan

2.1 Giả thuyết vé số may mắn

Giả thuyết vé số may mắn Lottery Ticket Hypothesis (LTH), được đề ra bởi Frankle và Carbin (2018), cho rằng: "Với 1 mạng neuron dày đặc feed-forward, được khởi tạo ngẫu nhiên chứa 1 mạng con "thưa" mà khi huấn luyện 1 cách độc lập khỏi khởi tạo gốc, có độ chính xác tập test cao hơn hoặc bằng mạng gốc với cùng số lần lặp" [2]. Vấn đề lớn nhất mà các bài báo về LTH nghiên cứu là về thuật toán để tìm ra tờ vé số may mắn này. Bài báo đầu tiên đề xuất ra LTH sử dụng thuật toán IMP. Phương pháp này dù chính xác nhưng đòi hỏi nhiều tài nguyên tính toán và thời gian. Bài luận này sẽ nghiên cứu và so sánh các phương pháp tìm kiếm vé số, đồng thời cải tiến 1 trong những phương pháp.

2.2 Phân loại thuật toán

Dựa trên các nghiên cứu hiện nay, các thuật toán tìm kiếm vé trúng thưởng có thể được phân nhóm dựa trên thời điểm thực hiện pruning và cơ chế xác định tầm quan trọng của các kết nối neuron.

2.2.1 Nhóm cắt tỉa sau kết nối

Đây là cách tiếp cận nguyên bản của LTH, yêu cầu mô hình phải trải qua quá trình huấn luyện để xác định các trọng số quan trọng.

IMP: Là phương pháp nền tảng dùng để xác định các mạng con có khả năng huấn luyện độc lập. Thuật toán thực hiện chu kỳ huấn luyện - cắt tỉa -

đặt lại (reset) trọng số về giá trị khởi tạo ban đầu [2]. Mặc dù IMP đạt hiệu suất cao ở mức độ thưa cực đại, chi phí tính toán của nó rất lớn do phải huấn luyện lại mạng nhiều lần.

2.2.2 Nhóm cắt tỉa giai đoạn sớm

Nhóm này tìm cách khắc phục nhược điểm về chi phí của IMP bằng cách xác định cấu trúc mạng con ngay khi quá trình huấn luyện bắt đầu.

EB Tickets: Thuật toán này phát hiện ra rằng các kết nối neurom quan trọng của mạng nơ-ron thường xuất hiện và ổn định rất sớm trong quá trình huấn luyện. Thay vì huấn luyện đến khi hội tụ, EB sử dụng các phương pháp giảm thời gian huấn luyện như dừng sớm (early stopping) kết hợp với biến có độ chính xác thấp (low-precision). Sự xuất hiện của vé EB được xác định thông qua metric "khoảng cách mặt nạ" (mask distance) giữa các epoch liên tiếp [12].

2.2.3 Nhóm cắt tỉa tại thời điểm khởi tạo

- GraSP: Khác với các phương pháp khác bảo toàn giá trị hàm mất mát, GraSP tập trung vào việc bảo toàn động lực huấn luyện bằng cách giữ lại các trọng số đóng góp nhiều nhất vào chuẩn gradient (gradient norm). Điều này giúp duy trì dòng chảy thông tin qua các lớp mạng ngay cả khi độ thưa (sparsity) rất cao [11].
- Iterative Synaptic Flow Pruning (SynFlow): Đây là thuật toán bất khả tri dữ liệu (data-agnostic), thực hiện cắt tỉa mà không cần nhìn vào dữ liệu huấn luyện. SynFlow hoạt động bằng cách bảo toàn tổng dòng chảy của sức mạnh khớp thần kinh xuyên suốt mạng nơ-ron thông qua chỉ số tích các trị tuyệt đối trọng số. Cơ chế lặp lại của SynFlow giúp nó đạt được giới hạn nén tối đa mà không gây ra hiện tượng sụp đổ lớp (layer-collapse) [10].

2.2.4 Nhóm cắt tỉa hỗn hợp

Hybrid Pruning: Một phương pháp cắt tỉa mới trong đó phần lớn các trọng số không quan trọng (khoảng 60-80%) được loại bỏ trong một bước duy nhất (one-shot), sau đó thực hiện một giai đoạn tinh chỉnh dài hơn. Tiếp theo,

các chu kỳ cắt tỉa lặp lại theo cấp số nhân (geometric-like) được áp dụng cho các trọng số còn lại để đạt được độ thưa mục tiêu với độ chính xác tối ưu [5].

Thuật toán	Thời điểm cắt tỉa	Cơ chế xác định tầm quan trọng	Đặc điểm chính
IMP	Sau huấn luyện	Độ lớn trọng số (Magnitude)	Tiêu chuẩn vàng, chi phí cao
Early-Bird	Giai đoạn sớm	Hệ số tỷ lệ BN (BN Scaling)	Tiết kiệm 5.8x – 10.7x năng lượng [12]
GraSP	Tại khởi tạo	Tín hiệu Gradient (Hessian-gradient)	Bảo toàn dòng chảy gradient
SynFlow	Tại khởi tạo	Dòng chảy neuron (Synaptic Flow)	Không cần dữ liệu, tránh sụp đổ lớp
Hybrid	Hỗn hợp	Độ lớn lặp lại kết hợp tinh chỉnh	Kết hợp ưu điểm One-shot và Iterative pruning

Table 1: So sánh các thuật toán cắt tỉa mạng nơ-ron

Chapter 3

Phương pháp

3.1 Thuật toán baseline

3.1.1 IMP

IMP là thuật toán nền tảng được Frankle và Carbin (2019) đề xuất để chứng minh LTH. IMP hoạt động dựa trên giả định rằng các trọng số có độ lớn (magnitude) nhỏ nhất là ít quan trọng nhất với hiệu suất mô hình [1, p.1]. Khác với phương pháp cắt tỉa 1 lần (one-shot pruning), IMP thực hiện việc loại bỏ trọng số qua nhiều chu kỳ huấn luyện và đặt lại giá trị, giúp "khử nhiễu" quá trình xác định tầm quan trọng của các kết nối neuron.

Xét một mạng neuron $f(x; \theta)$ với các tham số khởi tạo ban đầu $\theta_0 \sim D_\theta$. Một mạng con được xác định bởi một mặt nạ nhị phân (binary mask) $m \in \{0, 1\}^{|\theta|}$, có cùng kích thước với bộ tham số θ . Mục tiêu của thuật toán IMP là tìm kiếm mặt nạ m sao cho mạng con $f(x, m \odot \theta_0)$ đạt được hiệu suất tối ưu, trong đó $\odot$ là phép nhân Hadamard. Mặt nạ m tối ưu này thoả mãn các điều kiện:

- Độ thưa: $\|m\|_0 \ll |\theta|$ - số lượng tham số còn lại rất ít.
- Độ chính xác: $a' \geq a$ - độ chính xác của mạng con không thua kém mạng lớn gốc.
- Tốc độ hội tụ: $j' \leq j$ - số vòng lặp để hàm mất mát đạt cực tiểu không lớn hơn mạng gốc.

Ngoài ra, thuật toán này còn áp dụng cơ chế đặt lại (resetting) và late resetting. Sau mỗi vòng cắt tỉa, các trọng số còn sót lại phải được đặt lại (reset) về giá trị khởi tạo ban đầu θ_0 thay vì tiếp tục huấn luyện từ giá trị đã tối ưu. Bước

này đã được chứng minh thực nghiệm trong bài báo LTH gốc [3, p. 2], và được chứng minh một cách rõ ràng hơn trong [13].

Nghiên cứu này đã giải mã cơ chế đặt lại và phát hiện ra rằng yếu tố then chốt quyết định hiệu suất của một "vé trúng thưởng" không nằm ở giá trị độ lớn cụ thể, mà nằm ở dấu (sign) của trọng số tại thời điểm khởi tạo. Việc đặt lại trọng số về θ_0 đảm bảo các kết nối neuron bắt đầu trong cùng một "góc phần tư dấu" mà chúng đã chiếm giữ tại thời điểm khởi tạo, nơi mà bộ tối ưu hóa (optimizer) có thể dễ dàng điều hướng. Thực nghiệm cho thấy nếu giữ nguyên dấu ban đầu nhưng thay thế giá trị trọng số bằng một hằng số $\pm\alpha$, mạng con vẫn có thể đạt được độ chính xác cao tương đương với việc đặt lại về giá trị θ_0 chính xác. Điều này gợi ý rằng các bộ tối ưu hóa gặp khó khăn khi phải vượt qua "rào cản số không" để đổi dấu trọng số trong quá trình huấn luyện mạng thưa.

Mặc dù đặt lại về thời điểm $k = 0$ hoạt động tốt trên các tập dữ liệu nhỏ như MNIST, cơ chế này thường thất bại khi áp dụng cho các kiến trúc sâu và phức tạp hơn như ResNet-50 hoặc VGG-19 trên ImageNet. Frankle và cộng sự (2019) đã đề xuất kỹ thuật Late Resetting (hoặc Weight Rewinding), trong đó trọng số không được đặt lại về 0 mà về giá trị tại một thời điểm k rất sớm trong quá trình huấn luyện (thường là khoảng 0.1% đến 7% tổng thời gian) [3]. Cơ sở lý thuyết cho hiện tượng này là tính ổn định (stability) [13] đối với nhiễu của thuật toán hạ gradient ngẫu nhiên (SGD). Một mạng con được coi là ổn định nếu các phiên bản huấn luyện khác nhau của nó (với các hạt giống ngẫu nhiên khác nhau) đều hội tụ về cùng một lòng chảo (basin) tối ưu [3]. Việc quay ngược về thời điểm $k > 0$ giúp mạng con vượt qua giai đoạn biến động mạnh lúc ban đầu và đạt được sự kết nối neuron tuyến tính (linear mode connectivity), từ đó đảm bảo hiệu suất không bị suy giảm khi độ thưa tăng cao [9, sec. 4].

Thuật toán IMP có thể được biểu diễn như sau:

Input: Network $f(x; \theta)$, target pruning rate P , number of rounds n

Output: Winning ticket (θ_0, m)

Initialize random parameters θ_0 and mask $m = [1, \dots, 1]$;

Store initial parameters $\theta_{\text{orig}} = \theta_0$ (or θ_k if using Late Resetting);

for $i = 1$ **to** n **do**

 Train subnetwork $f(x; m \odot \theta_0)$ until convergence;

 Identify $p\%$ weights with smallest magnitude $|\theta|$;

 Update mask m : set selected weight positions to 0;

 Reset surviving weights: $\theta_0 = \theta_{\text{orig}} \odot m$;

end

return m, θ_{orig}

Algorithm 1: Iterative Magnitude Pruning (IMP)

3.1.2 EB

Thuật toán *Early-Bird Tickets* (EB Tickets), được đề xuất bởi You et al. [12], xuất phát từ quan sát rằng các "vé số may mắn" không nhất thiết phải chờ đến khi mô hình được huấn luyện hoàn toàn mới xuất hiện. Thay vào đó, cấu trúc kết nối quan trọng của mạng đã hình thành ổn định từ rất sớm trong quá trình huấn luyện, thường chỉ sau 6%–20% tổng số epoch. Dựa trên nhận xét này, EB Tickets đề xuất một chiến lược huấn luyện hiệu quả gồm hai giai đoạn (i) xác định sớm mạng con quan trọng bằng các phương án huấn luyện chi phí thấp (learning rate lớn, huấn luyện độ chính xác thấp, kết hợp dừng sớm), và (ii) tiếp tục huấn luyện chỉ mạng con đó đến độ chính xác mục tiêu. Kết quả thực nghiệm cho thấy EB Train có thể tiết kiệm tới $5.8\times$ – $10.7\times$ năng lượng huấn luyện trong khi vẫn duy trì độ chính xác tương đương hoặc cao hơn so với huấn luyện toàn mô hình trên các kiến trúc VGG16, PreResNet101, ResNet18/50 và các bộ dữ liệu CIFAR-10/100, ImageNet.

Cho mạng nơ-ron dày đặc $f(\mathbf{x}, \boldsymbol{\theta})$ được khởi tạo ngẫu nhiên, trong đó $\boldsymbol{\theta} \in \mathbb{R}^d$ là tập tham số (trọng số). Huấn luyện mạng bằng Stochastic Gradient Descent (SGD) trên tập huấn luyện $\mathcal{D}$ ký hiệu $\boldsymbol{\theta}^{(t)}$ là trọng số tại epoch thứ t . Gọi T là tổng số epoch huấn luyện đầy đủ, và f_{acc} là độ chính xác kiểm thử đạt được sau T epoch. Với mỗi epoch t , ta thực hiện cắt tỉa kênh có cấu trúc (structured channel pruning) theo tỉ lệ $p \in (0, 1)$ dựa trên hệ số tỉ lệ (scaling

factor) $\mathbf{r}^{(t)}$ trong các lớp Batch Normalization (BN):

$$\mathbf{m}^{(t)} = \text{Prune}_p(\mathbf{r}^{(t)}) \in \{0, 1\}^d,$$

trong đó $\mathbf{m}^{(t)}$ là mặt nạ nhị phân, với $m_i^{(t)} = 0$ nếu kênh i nằm trong $p\%$ giá trị nhỏ nhất của $\mathbf{r}^{(t)}$, và $m_i^{(t)} = 1$ ngược lại. Mạng con tương ứng được ký hiệu là $f(\mathbf{x}; \mathbf{m}^{(t)} \odot \boldsymbol{\theta}^{(t)})$, trong đó $\odot$ là phép nhân theo từng phần tử (Hadamard product).

Khoảng cách mask. Để đo lường mức độ ổn định của cấu trúc kết nối giữa các epoch liên tiếp, ta định nghĩa *khoảng cách mask* (mask distance) giữa hai epoch t và $t - 1$ bằng khoảng cách Hamming chuẩn hoá giữa hai ticket mask:

$$\delta^{(t)} = \frac{1}{d} \sum_{i=1}^d \mathbf{1}[m_i^{(t)} \neq m_i^{(t-1)}] = \frac{\|\mathbf{m}^{(t)} - \mathbf{m}^{(t-1)}\|_1}{d}, \quad (3.1)$$

với $\delta^{(t)} \in [0, 1]$; giá trị nhỏ biểu thị cấu trúc kết nối ổn định qua các epoch.

Điều kiện xuất hiện EB Ticket. Gọi $\mathcal{Q}^{(t)} = (\delta^{(t-l+1)}, \delta^{(t-l+2)}, \dots, \delta^{(t)})$ là hàng đợi FIFO lưu l khoảng cách mask gần nhất. Một *Early-Bird Ticket* được xác định tại epoch t^* khi:

$$t^* = \min \left\{ t \mid \max_{s \in \mathcal{Q}^{(t)}} \delta^{(s)} < \varepsilon \right\}, \quad (3.2)$$

trong đó $\varepsilon > 0$ là ngưỡng ổn định (mặc định $\varepsilon = 0.1$) và $l = 5$. Điều kiện (3.2) yêu cầu l khoảng cách mask liên tiếp *đồng thời* nhỏ hơn ε , tránh trường hợp ổn định giả tạo tại giai đoạn đầu huấn luyện.

Giả thuyết EB Ticket. Tồn tại $t^* \ll T$ và mask $\mathbf{m}^{(t^*)}$ sao cho mạng con $f(\mathbf{x}; \mathbf{m}^{(t^*)} \odot \boldsymbol{\theta}^{(t^*)})$, sau khi được tiếp tục huấn luyện (retrain), đạt độ chính xác f'_{acc} thoả:

$$f'_{\text{acc}} \gtrsim f_{\text{acc}}, \quad \text{với } t^*/T \ll 1 \text{ và } \|\mathbf{m}^{(t^*)}\|_0 \ll d. \quad (3.3)$$

Nói cách khác, EB Ticket vừa *xuất hiện sớm* vừa *thưa*, đảm bảo tiết kiệm cả chi phí tìm kiếm lẫn chi phí huấn luyện lại.

Đầu vào : Mạng $f(\mathbf{x}; \boldsymbol{\theta})$; tỉ lệ cắt tỉa p ; ngưỡng ổn định ε ; độ dài hàng đợi l ; số epoch tối đa $T_{\max}$

Đầu ra : Early-Bird Ticket $f(\mathbf{x}; \mathbf{m}^{(t^*)} \odot \boldsymbol{\theta}^{(t^*)})$

Khởi tạo trọng số $\boldsymbol{\theta}^{(0)}$, hệ số BN $\mathbf{r}^{(0)}$

Khởi tạo hàng đợi FIFO $\mathcal{Q} \leftarrow \emptyset$ (độ dài tối đa l)

$t \leftarrow 1$

while $t \leq T_{\max}$ **do**

 Cập nhật $\boldsymbol{\theta}^{(t)}, \mathbf{r}^{(t)}$ bằng một bước SGD trên $\mathcal{D}$

 ▷ Cắt tỉa có cấu trúc để thu ticket mask

$\mathbf{m}^{(t)} \leftarrow \text{Prune}_p(\mathbf{r}^{(t)})$

 ▷ Tính khoảng cách mask so với epoch trước

if $t > 1$ **then**

$\delta^{(t)} \leftarrow \frac{\|\mathbf{m}^{(t)} - \mathbf{m}^{(t-1)}\|_1}{d}$

 Thêm $\delta^{(t)}$ vào $\mathcal{Q}$ (loại bỏ phần tử cũ nhất nếu $|\mathcal{Q}| > l$)

end

 ▷ Kiểm tra điều kiện xuất hiện EB Ticket

if $|\mathcal{Q}| = l$ **và** $\max(\mathcal{Q}) < \varepsilon$ **then**

$t^* \leftarrow t$

return $f(\mathbf{x}; \mathbf{m}^{(t^*)} \odot \boldsymbol{\theta}^{(t^*)})$

end

$t \leftarrow t + 1$

end

return $f(\mathbf{x}; \mathbf{m}^{(T_{\max})} \odot \boldsymbol{\theta}^{(T_{\max})})$ ▷ Fallback: trả về mask cuối cùng nếu chưa hội tụ

Algorithm 2: Tìm kiếm Early-Bird Ticket (EB Search)

Sau khi EB Ticket được xác định tại epoch t^* , giai đoạn huấn luyện lại (retraining) tiếp tục với mạng con $f(\mathbf{x}; \mathbf{m}^{(t^*)} \odot \boldsymbol{\theta}^{(t^*)})$, kế thừa trọng số $\boldsymbol{\theta}^{(t^*)}$ thay vì khởi tạo lại từ đầu. Toàn bộ quy trình EB Train do đó chỉ yêu cầu huấn luyện đầy đủ trong $t^* + (T - t^*)_{\text{retrain}}$ epoch, trong đó $t^* \approx 6\%-20\% \cdot T$, dẫn đến tiết kiệm đáng kể về tổng chi phí tính toán và năng lượng so với các phương pháp cắt tỉa truyền thống yêu cầu huấn luyện mô hình đầy đủ hoàn toàn trước khi cắt tỉa.

3.1.3 GraSP

GraSP (Gradient Signal Preservation – Bảo toàn tín hiệu gradient) là một thuật toán tỉa mạng trước huấn luyện (*pruning at initialization*) được Wang et al. [11] đề xuất nhằm khắc phục hạn chế của tiêu chí *connection sensitivity* trong phương pháp SNIP. Tác giả chỉ ra khi tỉa một phần lớn trọng số, chuẩn gradient của mạng bị suy giảm đáng kể, làm chậm hoặc cản trở quá trình tối ưu hóa về sau. Thay vì bảo toàn giá trị hàm mất mát tại thời điểm khởi tạo—vốn không mang nhiều ý nghĩa vì mạng chưa được huấn luyện—GraSP trực tiếp tìm kiếm mặt nạ tỉa sao cho chuẩn gradient sau khi tỉa bị suy giảm ít nhất có thể.

Xét mạng nơ-ron $f(\mathbf{x}; \boldsymbol{\theta})$ được tham số hóa bởi $\boldsymbol{\theta} \in \mathbb{R}^d$ với giá trị khởi tạo $\boldsymbol{\theta}_0$. Cho tập huấn luyện $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ và hàm mất mát kinh nghiệm

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i). \quad (3.4)$$

Một *mặt nạ tỉa* (pruning mask) là vector nhị phân $\mathbf{m} \in \{0, 1\}^d$, trong đó $m_q = 0$ chỉ ra rằng trọng số thứ q bị loại bỏ. Bài toán tỉa trước huấn luyện được phát biểu là

$$\min_{\mathbf{m} \in \{0, 1\}^d} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f(\mathbf{x}; \mathcal{A}(\mathbf{m}, \boldsymbol{\theta}_0)), y)] \quad \text{s.t.} \quad \frac{\|\mathbf{m}\|_0}{d} = 1 - p, \quad (3.5)$$

trong đó $p \in (0, 1)$ là *tỷ lệ tỉa* mong muốn và $\mathcal{A}$ là thuật toán huấn luyện (ví dụ: SGD).

Tiêu chí tỉa dựa trên bảo toàn gradient. Tại điểm khởi tạo $\boldsymbol{\theta}_0$, mức độ giảm hàm mất mát tại mỗi bước gradient descent được đặc trưng bởi đạo hàm có hướng:

$$\Delta \mathcal{L}(\boldsymbol{\theta}) = \nabla \mathcal{L}(\boldsymbol{\theta})^\top \nabla \mathcal{L}(\boldsymbol{\theta}) = \|\nabla \mathcal{L}(\boldsymbol{\theta})\|^2. \quad (3.6)$$

GraSP mô hình hóa thao tác tỉa như một nhiễu loạn $\boldsymbol{\delta} \in \mathbb{R}^d$ tác động lên $\boldsymbol{\theta}_0$ (trong đó $\delta_q = -\theta_{0,q}$ nếu trọng số q bị tỉa, và $\delta_q = 0$ ngược lại). Sử dụng xấp

xĩ Taylor bậc nhất, sự thay đổi của $\Delta \mathcal{L}$ sau khi tĩa được ước lượng là:

$$\begin{aligned} S(\boldsymbol{\delta}) &= \Delta \mathcal{L}(\boldsymbol{\theta}_0 + \boldsymbol{\delta}) - \Delta \mathcal{L}(\boldsymbol{\theta}_0) \\ &\approx 2 \boldsymbol{\delta}^\top \underbrace{\nabla^2 \mathcal{L}(\boldsymbol{\theta}_0) \nabla \mathcal{L}(\boldsymbol{\theta}_0)}_{=: \mathbf{Hg}} + \mathcal{O}(\|\boldsymbol{\delta}\|_2^2), \end{aligned} \quad (3.7)$$

trong đó $\mathbf{Hg} = \mathbf{Hg}$ là tích vector-Hessian (Hessian-gradient product), $\mathbf{H} = \nabla^2 \mathcal{L}(\boldsymbol{\theta}_0)$ và $\mathbf{g} = \nabla \mathcal{L}(\boldsymbol{\theta}_0)$. Tích $\mathbf{Hg}$ có thể tính hiệu quả mà không cần xây dựng tường minh ma trận Hessian, nhờ phép vi phân tự động bậc hai.

Điểm quan trọng của từng trọng số. Thay $\boldsymbol{\delta} = -\boldsymbol{\theta}_0$ (tức là tĩa toàn bộ), mỗi thành phần của điểm quan trọng vector hóa được xác định bởi:

$$\boxed{\mathbf{S}(-\boldsymbol{\theta}_0) = -\boldsymbol{\theta}_0 \odot \mathbf{Hg}}, \quad (3.8)$$

trong đó $\odot$ là tích Hadamard (nhân theo từng phần tử). Nếu $S_q(-\theta_{0,q}) < 0$, việc tĩa trọng số q sẽ *làm giảm* luồng gradient; ngược lại nếu $S_q > 0$, việc tĩa sẽ *tăng* luồng gradient. Do đó, *điểm quan trọng càng cao thì trọng số càng ít quan trọng* đối với việc duy trì tốc độ học.

Xây dựng mặt nạ tĩa. Với tỷ lệ tĩa p , GraSP loại bỏ $\lfloor p \cdot d \rfloor$ trọng số có điểm quan trọng *cao nhất* (những trọng số mà việc tĩa ít làm hại nhất đến luồng gradient):

$$\tau = \text{Percentile}(\mathbf{S}(-\boldsymbol{\theta}_0), p), \quad m_q = \mathbf{1}[S_q(-\theta_{0,q}) < \tau]. \quad (3.9)$$

Mạng con thừa thu được $f(\mathbf{x}; \mathbf{m} \odot \boldsymbol{\theta}_0)$ sau đó được huấn luyện từ đầu trên $\mathcal{D}$.

Tính tích Hessian–gradient Thay vì xây dựng tường minh ma trận Hessian $\mathbf{H} \in \mathbb{R}^{d \times d}$ (tồn $\mathcal{O}(d^2)$ bộ nhớ), GraSP sử dụng tính toán vi phân tự động bậc hai để thu được $\mathbf{Hg} = \mathbf{Hg}$ với chi phí $\mathcal{O}(d)$. Cụ thể, xét biểu thức vô hướng $\mathbf{g}^\top \text{sg}(\mathbf{g})$ trong đó $\text{sg}(\cdot)$ là toán tử *stop-gradient*. Gradient của biểu thức này theo $\boldsymbol{\theta}$ chính là $\mathbf{Hg}$:

$$\nabla_{\boldsymbol{\theta}} [\mathbf{g}(\boldsymbol{\theta})^\top \text{sg}(\mathbf{g}(\boldsymbol{\theta}))] = \mathbf{H}(\boldsymbol{\theta}) \mathbf{g}(\boldsymbol{\theta}) = \mathbf{Hg}. \quad (3.10)$$

Thuật này cho phép tính $\mathbf{Hg}$ với độ phức tạp tương đương một lần lan truyền ngược bổ sung, không đòi hỏi bộ nhớ thêm cho Hessian.

Input: Mini-batch $\mathcal{D}_b$; mạng f với tham số khởi tạo $\boldsymbol{\theta}_0$; hàm mất mát ℓ

Output: Tích Hessian–gradient $\mathbf{Hg} \in \mathbb{R}^d$

Tính mất mát: $\mathcal{L}(\boldsymbol{\theta}_0) \leftarrow \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_b} [\ell(f(\mathbf{x}; \boldsymbol{\theta}_0), y)]$

Tính gradient bậc một: $\mathbf{g} \leftarrow \nabla_{\boldsymbol{\theta}_0} \mathcal{L}(\boldsymbol{\theta}_0)$

Tính tích Hessian–gradient bằng vi phân tự động bậc hai:

$\mathbf{Hg} \leftarrow \nabla_{\boldsymbol{\theta}_0} [\mathbf{g}^\top \text{sg}(\mathbf{g})]$ // theo phương trình (3.10)

return $\mathbf{Hg}$

Algorithm 3: Tính tích Hessian–gradient ($\mathbf{Hg}$)

Kết hợp bước tính điểm quan trọng (3.8) và xây dựng mặt nạ (3.9), thuật toán GraSP hoàn chỉnh được trình bày trong Thuật toán 4.

Input: Tỷ lệ tỉa $p \in (0, 1)$; tập dữ liệu huấn luyện $\mathcal{D}$; mạng f với tham số khởi tạo θ_0

Output: Mạng thưa đã huấn luyện $f(\mathbf{x}; \mathbf{m} \odot \theta)$

```
// Bước 1: Lấy mẫu mini-batch
Lấy ngẫu nhiên một mini-batch  $\mathcal{D}_b = \{(\mathbf{x}_i, y_i)\}_{i=1}^b \sim \mathcal{D}$ 

// Bước 2: Tính tích Hessian--gradient
 $\mathbf{Hg} \leftarrow \text{HessianGradientProduct}(\mathcal{D}_b, f, \theta_0)$ 

// Bước 3: Tính điểm quan trọng theo pt. (3.8)
for  $q = 1, \dots, d$  do
     $S_q \leftarrow -\theta_{0,q} \cdot (\mathbf{Hg})_q$ 
end

// Bước 4: Xác định ngưỡng tỉa
 $\tau \leftarrow \text{Percentile}(\{S_q\}_{q=1}^d, p)$ 

// Bước 5: Tạo mặt nạ nhị phân
for  $q = 1, \dots, d$  do
    if  $S_q < \tau$  then
         $m_q \leftarrow 1$ 
        // Giữ lại trọng số
    else
         $m_q \leftarrow 0$ 
        // Tỉa trọng số
    end
end

// Bước 6: Huấn luyện mạng con thưa từ đầu
Huấn luyện  $f(\mathbf{x}; \mathbf{m} \odot \theta)$  trên  $\mathcal{D}$  từ khởi tạo  $\mathbf{m} \odot \theta_0$  cho đến hội tụ
return  $f(\mathbf{x}; \mathbf{m} \odot \theta)$ 
```

Algorithm 4: Bảo toàn tín hiệu gradient (GraSP)

3.1.4 Synflow

Xét một mạng nơ-ron truyền thẳng (*feedforward*) $f(\mathbf{x}; \theta)$ gồm L tầng có trọng số. Ký hiệu $\theta = \{\theta^{[l]}\}_{l=1}^L$ là tập hợp tất cả tham số có thể tỉa, trong đó $\theta^{[l]} \in \mathbb{R}^{N^{[l]}}$ là véc-tơ tham số của tầng thứ l và $N^{[l]} = |\theta^{[l]}|$ là số lượng tham số

tương ứng. Tổng số tham số của mạng là $N = \sum_{l=1}^L N^{[l]}$.

Định nghĩa 3.1 (Tỉ lệ nén [10]) *Tỉ lệ nén $\rho \geq 1$ được định nghĩa là tỉ số giữa số tham số ban đầu và số tham số còn lại sau khi tỉa:*

$$\rho = \frac{N}{\#\{\text{tham số còn lại}\}}.$$

Tỉ lệ nén tối đa khả thi (không gây sụp đổ tầng) là $\rho_{\max} = N/L$, ứng với việc giữ lại đúng một tham số mỗi tầng.

Định nghĩa 3.2 (Mặt nạ tỉa thưa [10]) *Mặt nạ nhị phân $\boldsymbol{\mu} = \{\boldsymbol{\mu}^{[l]}\}_{l=1}^L$, $\boldsymbol{\mu}^{[l]} \in \{0, 1\}^{N^{[l]}}$, xác định tập tham số được giữ lại. Mạng sau khi tỉa được ký hiệu $f(\mathbf{x}; \boldsymbol{\mu} \odot \boldsymbol{\theta}_0)$, với $\boldsymbol{\theta}_0$ là giá trị tham số tại khởi tạo và $\odot$ là tích Hadamard.*

Định nghĩa 3.3 (Synaptic saliency [10]) *Với hàm mục tiêu vô hướng $\mathcal{R}$ là hàm của đầu ra mạng, điểm synaptic saliency của tham số θ_i được định nghĩa là*

$$\mathcal{S}(\theta_i) = \frac{\partial \mathcal{R}}{\partial \theta_i} \cdot \theta_i. \quad (3.11)$$

Đây là tích Hadamard giữa gradient của $\mathcal{R}$ và véc-tơ tham số, tổng quát hóa nhiều tiêu chí điểm quan trọng kinh điển (Skeletonization, SNIP, Taylor-FO, ...).

Định lý bảo toàn và hệ quả

Nền tảng lý thuyết của SynFlow dựa trên hai định lý bảo toàn sau, chứng minh rằng tổng điểm synaptic saliency được bảo toàn qua từng nơ-ron và toàn mạng [10].

Định lý 3.1 (Bảo toàn theo nơ-ron [10]) *Với mạng truyền thẳng có hàm kích hoạt liên tục và thuần nhất bậc nhất $\boldsymbol{\phi}(x) = \boldsymbol{\phi}'(x)x$ (ví dụ: ReLU, Leaky ReLU, tuyến tính), tổng điểm saliency của các tham số đến nơ-ron ẩn j bằng tổng điểm saliency của các tham số đi ra từ nơ-ron đó:*

$$\mathcal{S}_j^{\text{in}} = \left\langle \frac{\partial \mathcal{R}}{\partial \boldsymbol{\theta}_j^{\text{in}}}, \boldsymbol{\theta}_j^{\text{in}} \right\rangle = \left\langle \frac{\partial \mathcal{R}}{\partial \boldsymbol{\theta}_j^{\text{out}}}, \boldsymbol{\theta}_j^{\text{out}} \right\rangle = \mathcal{S}_j^{\text{out}}.$$

Định lý 3.2 (Bảo toàn theo mạng [10]) *Tổng điểm saliency trên bất kỳ tập tham số nào tách biệt hoàn toàn các nơ-ron đầu vào $\mathbf{x}$ khỏi các nơ-ron đầu ra $\mathbf{y}$*

đều bằng nhau và bằng

$$\left\langle \frac{\partial \mathcal{R}}{\partial \mathbf{x}}, \mathbf{x} \right\rangle = \left\langle \frac{\partial \mathcal{R}}{\partial \mathbf{y}}, \mathbf{y} \right\rangle.$$

Hệ quả – Nguyên nhân gây sụp đổ tầng [10]. Từ Định lý 3.2, tổng điểm saliency trên tầng l của một mạng fully-connected đơn giản là hằng số với mọi l . Do đó, điểm trung bình của tầng tỉ lệ nghịch với kích thước tầng:

$$\langle \mathcal{S}_i^{[l]} \rangle = \frac{C}{N^{[l]}},$$

với C là hằng số. Điều này giải thích tại sao các thuật toán tỉa đơn lần (*single-shot*) như SNIP hay GraSP ưu tiên loại bỏ tham số của tầng *lớn nhất*, dẫn đến sụp đổ tầng khi tỉ lệ nén đủ cao [10].

Điểm Synaptic Flow và hàm mục tiêu SynFlow

Để tránh sụp đổ tầng, SynFlow yêu cầu điểm tham số phải (i) *dương*, (ii) *bảo toàn theo tầng*, và (iii) *được tính lặp lại*. Ba điều kiện này được đảm bảo bởi hàm mục tiêu không dùng dữ liệu sau [10]:

$$\mathcal{R}_{\text{SF}}(\boldsymbol{\theta}) = \mathbf{1}^\top \left(\prod_{l=1}^L |\boldsymbol{\theta}^{[l]}| \right) \mathbf{1}, \quad (3.12)$$

trong đó $\mathbf{1}$ là véc-tơ toàn số 1 có kích thước phù hợp, $|\cdot|$ lấy giá trị tuyệt đối theo từng phần tử, và tích là tích ma trận qua các tầng. Đây chính là ℓ_1 -*path norm* của mạng.

Thay (3.12) vào (3.11), ta thu được *điểm Synaptic Flow* của tham số $w_{ij}^{[l]}$ [10]:

$$\mathcal{S}_{\text{SF}}(w_{ij}^{[l]}) = \left[\mathbf{1}^\top \prod_{k=l+1}^N |\mathbf{W}^{[k]}| \right]_i \cdot |w_{ij}^{[l]}| \cdot \left[\prod_{k=1}^{l-1} |\mathbf{W}^{[k]}| \mathbf{1} \right]_j. \quad (3.13)$$

Biểu thức (3.13) cho thấy điểm SynFlow là tổng tích các trọng số tuyệt đối dọc theo mọi đường đi từ đầu vào đến đầu ra mà đi qua $w_{ij}^{[l]}$, tổng quát hóa điểm độ lớn đơn thuần $|w_{ij}^{[l]}|$ bằng cách tính đến sự tương tác liên tầng [10]

Điều kiện đủ để đạt Nén Tối Hạn Cực Đại

Định lý 3.3 (Tỉa lặp với điểm dương và bảo toàn đạt Nén Tối Hạn Cực Đại)

Nếu một thuật toán tỉa với global masking gán điểm dương thỏa mãn bảo toàn theo tầng, đồng thời lượng tỉa (prune size) tại mỗi vòng lặp luôn nhỏ hơn nghiêm ngặt so với kích thước cắt (cut size) – tức tổng điểm của cả một tầng – khi điều này khả thi, thì thuật toán đó thỏa mãn tiên đề Nén Tối Hạn Cực Đại [10].

Bộ ba điều kiện của Định lý 3.3 – điểm dương, bảo toàn, và tỉa lặp – được SynFlow đáp ứng đồng thời qua hàm mục tiêu (3.12) kết hợp với lịch trình nén theo hàm mũ $\rho_k = \rho^{k/n}$, với n là tổng số vòng lặp tỉa.

Đầu vào: Mạng $f(\mathbf{x}; \boldsymbol{\theta}_0)$; tỉ lệ nén ρ ; số vòng lặp n

Đầu ra : Mạng tỉa thừa $f(\mathbf{x}; \boldsymbol{\mu} \odot \boldsymbol{\theta}_0)$

Chuyển mạng sang chế độ eval (vô hiệu hóa Batch Normalization)

Khởi tạo mặt nạ $\boldsymbol{\mu} \leftarrow \mathbb{1}$

▷ giữ lại toàn bộ tham số

for $k = 1, 2, \dots, n$ **do**

$\boldsymbol{\theta}_\mu \leftarrow \boldsymbol{\mu} \odot \boldsymbol{\theta}_0$

▷ che tham số đã tỉa

▷ Tính hàm mục tiêu SynFlow (không cần dữ liệu)

$\mathcal{R} \leftarrow \mathbb{1}^\top \left(\prod_{l=1}^L |\boldsymbol{\theta}_\mu^{[l]}| \right) \mathbb{1}$

▷ Tính điểm Synaptic Flow qua lan truyền ngược

$\mathbf{S} \leftarrow \frac{\partial \mathcal{R}}{\partial \boldsymbol{\theta}_\mu} \odot \boldsymbol{\theta}_\mu$

▷ Xác định ngưỡng theo lịch trình nén mũ

$\tau \leftarrow (1 - \rho^{-k/n})\text{-percentile}(\mathbf{S})$

▷ Cập nhật mặt nạ: giữ lại các tham số có điểm vượt ngưỡng

$\boldsymbol{\mu} \leftarrow \mathbb{1}[\mathbf{S} > \tau]$

end

return $f(\mathbf{x}; \boldsymbol{\mu} \odot \boldsymbol{\theta}_0)$

Algorithm 5: Iterative Synaptic Flow Pruning – SynFlow [10]

Bởi vì hàm mục tiêu $\mathcal{R}_{\text{SF}}$ là tích các ma trận trọng số tuyệt đối, giá trị của nó có thể tăng hoặc giảm theo hàm mũ của chiều sâu L , tiềm ẩn nguy cơ mất

ổn định số học với mạng rất sâu. Một biện pháp khắc phục đơn giản là chuẩn hóa từng tầng trước khi tính điểm, hoặc sử dụng dấu phẩy động độ chính xác kép (`float64`). Trên thực nghiệm, $n = 100$ vòng lặp với lịch trình mũ là đủ để SynFlow đạt Nén Tối Hạn Cực Đại trên tất cả các kiến trúc VGG và ResNet được khảo sát [10].

3.1.5 Hybrid

Các phân tích thực nghiệm trong tài liệu gốc [5] chỉ ra rằng không có một chiến lược cắt tỉa đơn lẻ nào là tối ưu trong mọi điều kiện: cắt tỉa một lần (*one-shot pruning*) đạt hiệu quả cao hơn ở các tỉ lệ thưa thớt thấp (dưới 80%), trong khi cắt tỉa lặp lại (*iterative pruning*) lại vượt trội ở các tỉ lệ thưa thớt cao hơn. Từ quan sát này, phương pháp **cắt tỉa lai** (*hybrid few-shot pruning*) được đề xuất như một chiến lược kết hợp nhằm tận dụng ưu điểm của cả hai phương pháp. Ý tưởng cốt lõi là loại bỏ phần lớn trọng số không cần thiết trong một bước lớn duy nhất giống one-shot, sau đó tinh chỉnh lại cấu trúc còn lại qua một chuỗi các bước lặp hình học nhỏ hơn. Cách tiếp cận này vừa tránh được chi phí tính toán lặp lại của iterative pruning ở giai đoạn đầu, vừa bảo toàn độ chính xác nhờ các bước tinh chỉnh dần dần ở giai đoạn sau, khi mà các trọng số còn lại ngày càng trở nên quan trọng hơn.

Cho một mạng nơ-ron $f(x, ; \boldsymbol{\theta})$ với tập trọng số $\boldsymbol{\theta} \in \mathbb{R}^W$, trong đó W là tổng số tham số có thể huấn luyện. Quá trình cắt tỉa được điều khiển bởi một **mặt nạ nhị phân** (binary mask) $\mathbf{m} \in \{0, 1\}^W$, trong đó $m_i = 0$ biểu thị tham số θ_i đã bị loại bỏ. Số lượng tham số còn lại sau khi áp dụng mặt nạ là $\|\mathbf{m}\|_0 = \sum_{i=1}^W m_i$. Tỉ lệ thưa thớt (*sparsity*) tại thời điểm bất kỳ được định nghĩa là:

$$s(\mathbf{m}) = 1 - \frac{\|\mathbf{m}\|_0}{W}. \quad (3.14)$$

Mục tiêu là đạt được tỉ lệ thưa thớt mục tiêu $p \in (0, 1)$, tức là $s(\mathbf{m}^*) = p$, đồng thời duy trì độ chính xác của mô hình ở mức cao nhất có thể.

Tiêu chí quan trọng theo độ lớn (magnitude criterion). Tại mỗi bước cắt tỉa, tầm quan trọng của tham số θ_i được ước lượng bằng giá trị tuyệt đối của nó:

$$\text{score}(\theta_i) = |\theta_i|. \quad (3.15)$$

Các tham số có điểm số thấp nhất sẽ bị loại bỏ, tức là bị gán $m_i = 0$.

Bước cắt tỉa một lần lớn (one-shot step). Cho tỉ lệ cắt tỉa one-shot $p_k \in (0, p)$ với $p_k = \alpha \cdot p$, trong đó $\alpha \in [0.6, 0.8]$ là hệ số phân bổ. Mặt nạ sau bước one-shot được xác định bởi:

$$m_i^{(0)} = \begin{cases} 0, & \text{nếu } |\theta_i| \leq \tau_{p_k} \\ 1, & \text{ngược lại,} \end{cases} \quad (3.16)$$

trong đó τ_{p_k} là ngưỡng được chọn sao cho tỉ lệ thừa thớt sau bước này bằng đúng p_k , tức là $s(\mathbf{m}^{(0)}) = p_k$. Một cách tương đương, τ_{p_k} chính là phân vị thứ p_k của phân phối $\{|\theta_i|\}_{i=1}^W$.

Số trọng số còn lại. Sau bước one-shot, số trọng số chưa bị cắt tỉa là:

$$W^{(0)} = \|\mathbf{m}^{(0)}\|_0 = W(1 - p_k). \quad (3.17)$$

Lịch trình cắt tỉa hình học lặp lại (iterative geometric schedule). Gọi $p_i \ll p_k$ là tỉ lệ cắt tỉa tại mỗi bước lặp, được áp dụng trên số trọng số *còn lại* tại bước đó. Sau n bước lặp hình học, số trọng số còn lại là:

$$W^{(n)} = W^{(0)} \cdot (1 - p_i)^n = W(1 - p_k)(1 - p_i)^n. \quad (3.18)$$

Mặt nạ tại bước lặp thứ n được cập nhật theo quy tắc:

$$m_i^{(n)} = \begin{cases} 0, & \text{nếu } m_i^{(n-1)} = 1 \text{ và } |\theta_i| \leq \tau_{p_i}^{(n)} \\ m_i^{(n-1)}, & \text{ngược lại,} \end{cases} \quad (3.19)$$

trong đó $\tau_{p_i}^{(n)}$ là ngưỡng phân vị thứ p_i tính *chỉ trên tập trọng số còn sống* $\{i : m_i^{(n-1)} = 1\}$.

Điều kiện dừng. Quá trình lặp dừng khi tỉ lệ thừa thớt đạt mục tiêu p :

$$s(\mathbf{m}^{(n^*)}) \geq p, \quad (3.20)$$

trong đó $n^* = \left\lceil \frac{\ln\left(\frac{1-p}{1-p_k}\right)}{\ln(1-p_i)} \right\rceil$ là số bước lặp tối thiểu cần thiết theo lịch trình hình học.

Tinh chỉnh dựa trên patience (patience-based fine-tuning). Sau mỗi bước cắt tỉa, mô hình được tinh chỉnh cho đến khi độ chính xác trên tập kiểm định không cải thiện thêm trong ρ epoch liên tiếp. Cụ thể, tại bước t của việc tinh chỉnh, thuật toán dừng sớm nếu:

$$\text{acc}_{\text{val}}^{(t)} \leq \max_{t' < t} \text{acc}_{\text{val}}^{(t')} + \delta \quad \text{trong } \rho \text{ epoch liên tiếp}, \quad (3.21)$$

trong đó $\delta \geq 0$ là ngưỡng cải thiện tối thiểu. Tham số patience cho bước one-shot được đặt là $\rho_k \approx 200$ epoch, còn với mỗi bước lặp là $\rho_i = \rho_k/20$.

Theo khuyến nghị thực nghiệm [5]: khi $p < 0.8$, nên đặt $p_i \approx 0.10$; khi $p \geq 0.8$, nên đặt $p_i \approx 0.02$. Hệ số α thường được chọn trong khoảng $[0.6, 0.8]$, tương ứng với việc loại bỏ 60%–80% tổng lượng cắt tỉa mục tiêu ngay trong bước đầu tiên.

3.2 Hybrid Improved

3.2.1 Điểm yếu của thuật toán Hybrid

Mặc dù Hybrid Pruning đã cho thấy hiệu quả vượt trội so với hai chiến lược riêng lẻ trong phần lớn các kịch bản, thuật toán gốc vẫn tồn tại ba điểm yếu cơ bản ảnh hưởng đến hiệu năng và khả năng tổng quát hóa.

Đầu tiên, thuật toán gốc khuyến nghị $r_k \in [0.60, 0.80]$ dựa trên kết quả grid search trên một tập kiến trúc và dataset hạn chế [5, Sec .5.5]. Không có nguyên tắc lý thuyết nào kết nối r_k với khả năng chịu đựng của mạng đối với việc loại bỏ trọng số đột ngột. Cùng một giá trị r_k được áp dụng cho ResNet và ViT, mặc dù hai kiến trúc này có bề mặt mất mát (loss landscape) rất khác nhau. Hệ quả là với các kiến trúc chưa được benchmark, người dùng phải thực

Đầu vào: Mô hình đã huấn luyện $f(\cdot; \theta)$; tỉ lệ mục tiêu p ; hệ số phân bố $\alpha \in [0.6, 0.8]$; tỉ lệ bước lặp p_i ; patience one-shot ρ_k ; patience lặp $\rho_i = \rho_k/20$; ngưỡng cải thiện δ ; tốc độ học $\eta = \eta_0/10$

Đầu ra: Mô hình đã cắt tỉa $f(\cdot; \theta \odot m^*)$ với $s(m^*) \approx p$

▷ Khởi tạo mặt nạ toàn một

$m \leftarrow \mathbf{1}^W$

▷ --- Giai đoạn 1: Bước cắt tỉa one-shot lớn ---

$p_k \leftarrow \alpha \cdot p$

$m \leftarrow \text{CắtTỉaTheoĐộLớn}(\theta, m, p_k)$ ▷ Loại bỏ $p_k \cdot W$ trọng số nhỏ nhất toàn cục

$\theta \leftarrow \text{TinhChỉnh}(f, \theta, m, \eta, \rho_k, \delta)$ ▷ Tinh chỉnh kéo dài với patience ρ_k

▷ --- Giai đoạn 2: Cắt tỉa hình học lặp lại ---

while $s(m) < p$ **do**

$m \leftarrow \text{CắtTỉaTheoĐộLớn}(\theta, m, p_i)$ ▷ Xem công thức (3.19)

$\theta \leftarrow \text{TinhChỉnh}(f, \theta, m, \eta, \rho_i, \delta)$ ▷ Tinh chỉnh ngắn với patience $\rho_i \ll \rho_k$

end

$m^* \leftarrow m$

return $f(\cdot; \theta \odot m^*)$

Algorithm 6: Cắt tỉa lai — Vòng lặp chính (1/3)

Hàm CắtTỉaTheoĐộLớn(θ, m, r):

Input: Trọng số θ ; mặt nạ hiện tại m ; tỉ lệ cắt tỉa $r \in (0, 1)$ trên số trọng số còn sống

Output: Mặt nạ m đã cập nhật

$\mathcal{A} \leftarrow \{i : m_i = 1\}$ ▷ Tập chỉ số trọng số còn sống

$k \leftarrow \lfloor r \cdot |\mathcal{A}| \rfloor$ ▷ Số trọng số cần loại bỏ bước này

Sắp xếp $\mathcal{A}$ theo $|\theta_i|$ tăng dần thành $[a_1, a_2, \dots]$

for $j = 1$ **to** k **do** $m_{a_j} \leftarrow 0$

return m

Algorithm 6: Cắt tỉa lai — Hàm CắtTỉaTheoĐộLớn (2/3)

Hàm TinhChỉnh($f, \theta, m, \eta, \rho, \delta$):

Input: Mô hình f ; trọng số θ ; mặt nạ m ; tốc độ học η ; patience ρ ; ngưỡng δ

Output: Trọng số θ^* tốt nhất quan sát được

$acc^* \leftarrow 0$; $\theta^* \leftarrow \theta$; $c \leftarrow 0$

repeat

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(f, \theta, m)$ ▷ Một bước SGD có mặt nạ

$\theta_i \leftarrow \theta_i \cdot m_i \forall i$ ▷ Tái áp dụng mặt nạ sau cập nhật

 Tính acc_{val} trên tập kiểm định

if $acc_{val} > acc^* + \delta$ **then**

$acc^* \leftarrow acc_{val}$; $\theta^* \leftarrow \theta$; $c \leftarrow 0$

else

$c \leftarrow c + 1$

end

until $c \geq \rho$

return θ^*

Algorithm 6: Cắt tỉa lại — Hàm TinhChỉnh với dừng sớm (3/3)

hiện grid search tốn kém, hoặc chấp nhận hiệu năng dưới mức tối ưu.

Thứ hai, quá trình tinh chỉnh sau bước one-shot dựa hoàn toàn vào task loss. Sau khi tỉa $r_k \times p$ trọng số trong một bước, mạng chịu một cú sốc cấu trúc lớn. Thuật toán gốc khôi phục hiệu năng thuần túy thông qua ≈ 200 epoch tinh chỉnh bằng task loss. Điều này có hai vấn đề. Thứ nhất, nó tốn kém về tính toán và không có tín hiệu bên ngoài nào hướng dẫn mạng về những biểu diễn (representation) nào cần khôi phục. Thứ hai, khi r_k lớn (gần 80%), mạng thường không hội tụ đủ trong τ_k epoch vì không gian tham số bị thu hẹp đột ngột quá nhiều.

Thứ ba, chuyển tiếp đột ngột từ giai đoạn one-shot sang iterative. Khi kết thúc giai đoạn one-shot, learning rate đã suy giảm về gần 0 theo lịch cosine (cosine learning rate schedule). Giai đoạn iterative geometric bắt đầu từ learning rate nhỏ η_{ft} mà không có giai đoạn khởi động (warm-up), tức là ngay sau mỗi bước tỉa nhỏ tiếp theo, gradient descent hoạt động trong một vùng rất hẹp quanh điểm hội tụ của giai đoạn trước. Điều này hạn chế khả năng tái tổ chức trọng số, đặc biệt ở tỉ lệ tỉa cao khi các trọng số còn lại cần thích nghi đáng kể với cấu trúc mạng mới.

3.2.2 Đề xuất cải tiến

Để giải quyết vấn đề thứ nhất, thay vì chọn r_k theo kinh nghiệm, ta muốn xác định r_k trực tiếp từ đặc tính của bề mặt mất mát trước khi tĩa. Ta thực hiện điều này dựa trên hàm mất mát: nếu bề mặt mất mát phẳng (độ cong thấp), mạng chịu được tĩa mạnh vì các trọng số còn lại dễ bù đắp cho phần đã bị loại bỏ; ngược lại, bề mặt nhọn đòi hỏi r_k nhỏ hơn.

Fisher Information Trace. Fisher Information Matrix (FIM) là

$$\mathbf{F} = \mathbb{E}_{(x,y) \sim p_{\text{data}}} [\nabla_{\theta} \log p(y|x, \theta) \nabla_{\theta} \log p(y|x, \theta)^{\top}]$$

Trace của $\mathbf{F}$ đo tổng độ cong của log-likelihood theo mọi hướng tham số. Thay vì tính FIM đầy đủ (chi phí $O(|\theta|^2)$), ta ước lượng trace thực nghiệm bằng chuẩn bình phương gradient:

$$\hat{\mathcal{F}} = \frac{1}{|\theta|} \cdot \frac{1}{B} \sum_{b=1}^B \|\nabla_{\theta} \mathcal{L}(x_b, y_b; \theta)\|^2 \quad (3.22)$$

trong đó B là số mini-batch dùng để ước lượng (trong thực nghiệm, $B = 10$ là đủ ổn định). Đây là phép tính chi phí thấp: chỉ cần một forward-backward pass, không cần nghịch đảo ma trận Hessian.

Ánh xạ Fisher trace sang r_k . Ta ánh xạ $\hat{\mathcal{F}}$ sang tỉ lệ one-shot thông qua hàm sigmoid:

$$r_k = \text{clip} \left(0.8 - 0.3 \cdot \sigma \left(\alpha \cdot \ln \frac{\hat{\mathcal{F}}}{\mathcal{F}_{\text{ref}}} \right), 0.50, 0.80 \right) \quad (3.23)$$

trong đó $\sigma(\cdot)$ là hàm sigmoid, $\mathcal{F}_{\text{ref}}$ là giá trị tham chiếu theo kiến trúc (ví dụ: 10^{-3} với ResNet/CIFAR), và α điều khiển độ nhạy. Khi $\hat{\mathcal{F}} \gg \mathcal{F}_{\text{ref}}$ (bề mặt nhọn), $\sigma(\cdot) \rightarrow 1$, và $r_k \rightarrow 0.5$; khi $\hat{\mathcal{F}} \ll \mathcal{F}_{\text{ref}}$ (bề mặt phẳng), $\sigma(\cdot) \rightarrow 0$, và $r_k \rightarrow 0.8$.

Để giải quyết vấn đề thứ hai, thay vì để task loss tự tìm lại biểu diễn tốt sau khi mạng bị tĩa mạnh, ta dùng chính mô hình dày đặc trước khi tĩa làm *teacher* để hướng dẫn mạng đã tĩa (student) khôi phục các biểu diễn quan trọng.

Hàm mất mát distillation. Trước bước one-shot, ta lưu ảnh chụp (snapshot) θ_0 của mô hình dày đặc. Sau khi tỉa tới tỉ lệ $r_k \cdot p$, ta tinh chỉnh mô hình đã tỉa θ_{pk} bằng hàm mất mát kết hợp:

$$\mathcal{L}_{\text{total}} = (1 - \lambda) \mathcal{L}_{\text{task}}(\theta_{pk}) + \lambda \mathcal{L}_{\text{KD}}(\theta_{pk}, \theta_0), \quad (3.24)$$

trong đó hàm distillation dựa trên KL-divergence giữa phân phối softmax nhiệt độ T của hai mô hình:

$$\mathcal{L}_{\text{KD}}(\theta_{pk}, \theta_0) = T^2 \cdot D_{\text{KL}}\left(\sigma\left(\frac{f_{\theta_0}(x)}{T}\right) \parallel \sigma\left(\frac{f_{\theta_{pk}}(x)}{T}\right)\right) \quad (3.25)$$

Hệ số T^2 chuẩn hóa gradient về cùng biên độ với $\mathcal{L}_{\text{task}}$.

Sau khi giai đoạn one-shot kết thúc, mạng đã khôi phục hiệu năng gần với mạng teacher. Tiếp tục dùng KD trong giai đoạn geometric sẽ kìm hãm các điều chỉnh trọng số nhỏ mà geometric pruning cần thực hiện — teacher đã không còn là mục tiêu phù hợp vì cấu trúc mạng đã khác. Do đó, $\mathcal{L}_{\text{KD}}$ chỉ được dùng khi $n = 0$ (bước one-shot), và bị tắt hoàn toàn với $n \geq 1$.

Với vấn đề thứ ba, ở cuối giai đoạn one-shot, cosine annealing đã đưa learning rate về gần bằng 0. Khi giai đoạn iterative bắt đầu và thực hiện bước tỉa đầu tiên, gradient descent chỉ hoạt động trong vùng cực kỳ hẹp quanh điểm hội tụ cũ. Điều này hạn chế khả năng các trọng số còn lại tái tổ chức để bù đắp cho phần vừa bị tỉa.

Để khắc phục điều này, tại mỗi bước iterative ($n \geq 1$), ta thay thế lịch cosine trần bằng một lịch kết hợp: warm-up tuyến tính trong E_w epoch đầu, tiếp theo bởi cosine annealing trong phần còn lại:

$$\eta(e) = \begin{cases} \eta_{\text{ft}} \cdot \frac{e+1}{E_w} & \text{nếu } e < E_w, \\ \eta_{\text{ft}} \cdot \frac{1}{2} \left(1 + \cos\left(\frac{\pi(e - E_w)}{E_{\text{max}} - E_w}\right) \right) & \text{nếu } e \geq E_w \end{cases} \quad (3.26)$$

Warm-up $E_w = 5$ epoch là đủ để cho trọng số còn lại không gian thích nghi với cấu trúc mạng mới trước khi learning rate giảm dần. Cơ chế này không phải là full rewinding (như trong) mà là một phiên bản nhẹ: chỉ áp dụng ở đầu mỗi bước iterative, không làm lại từ đầu lịch của giai đoạn ban đầu. Mã giả của thuật toán như sau:

Input: Mạng $\mathcal{M}$, dữ liệu $\mathcal{D}$, learning rate η_0

Output: Snapshot teacher θ_0

Huấn luyện $\mathcal{M}$ đến hội tụ với learning rate η_0

$\theta_0 \leftarrow \mathcal{M}$ // lưu snapshot teacher

Algorithm 7: Hybrid Pruning Cải tiến – Giai đoạn 1: Huấn luyện ban đầu

Input: Tỷ lệ tỉa mục tiêu p , độ nhạy Fisher α , tham chiếu $\mathcal{F}_{\text{ref}}$, tỉ lệ
iterative p_i

Output: Lịch trình tỉa $\mathcal{S}$

$\hat{\mathcal{F}} \leftarrow \frac{1}{|\theta|} \cdot \frac{1}{B} \sum_{b=1}^B \|\nabla_{\theta} \mathcal{L}(x_b, y_b; \theta)\|^2$

$r_k \leftarrow \text{clip}\left(0.8 - 0.3 \cdot \sigma\left(\alpha \ln\left(\hat{\mathcal{F}} / \mathcal{F}_{\text{ref}}\right)\right), 0.50, 0.80\right)$

$\mathcal{S} \leftarrow []$

$\tilde{p} \leftarrow 0$

$n \leftarrow 0$

while $p - \tilde{p} > \varepsilon$ **do**

if $n = 0$ **then**

$\Delta \leftarrow r_k \cdot p$

else

$\Delta \leftarrow \min(p_i(1 - \tilde{p}), p - \tilde{p})$

end

$\mathcal{S} \leftarrow \mathcal{S} \cup \{\Delta / (1 - \tilde{p})\}$

$\tilde{p} \leftarrow \tilde{p} + \Delta$

$n \leftarrow n + 1$

end

Algorithm 8: Hybrid Pruning Cải tiến – Giai đoạn 2: Xây dựng lịch trình
tỉa

Input: Mạng $\mathcal{M}$, snapshot teacher θ_0 , lịch trình tỉa $\mathcal{S}$, patience

τ_k, τ_i , temperature T , KD weight λ , warm-up epoch E_w

Output: Mạng đã tỉa $\mathcal{M}^*$

$\eta_{\text{ft}} \leftarrow \eta_0/10$

for $n \leftarrow 1$ **to** $|\mathcal{S}|$ **do**

$s_n \leftarrow \mathcal{S}[n]$

 Tỉa s_n trọng số có magnitude nhỏ nhất trong số trọng số còn lại

if $n = 1$ **then**

$\tau \leftarrow \tau_k$

$\mathcal{L}_{\text{eff}} \leftarrow (1 - \lambda) \mathcal{L}_{\text{task}} + \lambda T^2 D_{\text{KL}}(\sigma(f_{\theta_0}(x)/T) \parallel \sigma(f_{\mathcal{M}}(x)/T))$

else

$\tau \leftarrow \tau_i$

$\mathcal{L}_{\text{eff}} \leftarrow \mathcal{L}_{\text{task}}$

end

while *validation accuracy has not improved for τ epochs* **do**

if $n \geq 2$ **then**

$\eta \leftarrow \eta_{\text{ft}} \cdot \frac{\text{epoch}+1}{E_w}$

end

$\eta \leftarrow \text{cosine schedule}(\eta_{\text{ft}})$

 Huấn luyện 1 epoch với $\mathcal{L}_{\text{eff}}$ và η ; giữ lại mặt nạ tỉa

end

 Khôi phục checkpoint có độ chính xác tốt nhất

end

return $\mathcal{M}^*$

Algorithm 9: Hybrid Pruning Cải tiến – Giai đoạn 3: Tỉa và tinh chỉnh

Chapter 4

Thiết lập thử nghiệm

4.1 Bộ dữ liệu

Các thử nghiệm được thực hiện trên ba bộ dữ liệu phân loại ảnh tiêu chuẩn. Các bộ dữ liệu này được chọn vì chúng được sử dụng rộng rãi trong văn nghiên cứu về pruning mạng nơ-ron, cho phép so sánh trực tiếp với các công trình trước đây [3, 11, 10, 5].

4.1.1 MNIST

MNIST [8] là bộ dữ liệu nhận dạng chữ số viết tay gồm 70 000 ảnh thang độ xám kích thước 28×28 pixel, trong đó 60 000 ảnh dùng để huấn luyện và 10 000 ảnh dùng để kiểm tra. Bộ dữ liệu bao gồm 10 lớp tương ứng với các chữ số từ 0 đến 9.

Trong luận văn này, MNIST được sử dụng như một thiết lập kiểm chứng nhanh (*quick validation*) để xác nhận tính đúng đắn của các thuật toán pruning trước khi chạy trên các bộ dữ liệu phức tạp hơn. Bộ dữ liệu không yêu cầu tiền xử lý đặc biệt; ảnh chỉ được chuẩn hóa về khoảng $[0, 1]$ bằng cách chia cho 255.

4.1.2 CIFAR-10

CIFAR-10 [7] là bộ dữ liệu phân loại ảnh màu kích thước 32×32 pixel gồm 60 000 ảnh thuộc 10 lớp đối tượng (máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu thủy, xe tải), với 50 000 ảnh huấn luyện và 10 000 ảnh kiểm tra (mỗi lớp 5 000/1 000 ảnh).

CIFAR-10 là bộ dữ liệu tham chiếu chính trong luận văn vì đây là benchmark tiêu chuẩn được hầu hết các công trình IMP sử dụng.

Quá trình tiền xử lý bao gồm:

- **Huấn luyện:** Lật ngang ngẫu nhiên (random horizontal flip), cắt ngẫu nhiên sau padding 4 pixel (random crop with padding=4), chuẩn hóa với giá trị trung bình $\mu = (0.4914, 0.4822, 0.4465)$ và độ lệch chuẩn $\sigma = (0.2023, 0.1994, 0.2010)$.
- **Kiểm tra:** Chỉ chuẩn hóa (không tăng cường dữ liệu).

4.1.3 CIFAR-100

CIFAR-100 [7] có cùng định dạng và kích thước với CIFAR-10 (60 000 ảnh 32×32) nhưng được phân chia thành 100 lớp chi tiết hơn, mỗi lớp có 500 ảnh huấn luyện và 100 ảnh kiểm tra. Đây là bộ dữ liệu thách thức hơn vì không gian nhãn rộng hơn đáng kể.

CIFAR-100 được sử dụng để đánh giá khả năng mở rộng (scalability) của các phương pháp pruning khi độ phức tạp của tác vụ phân loại tăng lên. Quy trình tiền xử lý tương tự CIFAR-10, với giá trị chuẩn hóa riêng: $\mu = (0.5071, 0.4867, 0.4408)$ và $\sigma = (0.2675, 0.2565, 0.2761)$.

Table 2: Tổng quan các bộ dữ liệu sử dụng trong thực nghiệm.

Bộ dữ liệu	Lớp	Kích thước ảnh	Train	Test	Màu sắc	Vai trò
MNIST	10	28×28	60 000	10 000	Xám	Kiểm chứng nhanh
CIFAR-10	10	32×32	50 000	10 000	RGB	Benchmark chính
CIFAR-100	100	32×32	50 000	10 000	RGB	Đánh giá mở rộng

4.2 Kiến trúc mạng

Bốn kiến trúc mạng nơ-ron được sử dụng trong luận văn nhằm đánh giá hiệu quả của các thuật toán pruning trên nhiều quy mô mô hình khác nhau, từ mạng nhỏ gọn đến mạng sâu có dung lượng lớn.

4.2.1 ResNet-20

ResNet-20 [4] là phiên bản nhỏ nhất của họ ResNet được thiết kế đặc biệt cho bộ dữ liệu CIFAR. Mạng gồm 20 lớp tích chập với tổng cộng khoảng **270 000 tham số**, được tổ chức thành ba nhóm (stages) với lần lượt 16, 32 và 64 bộ lọc, mỗi nhóm gồm $n = 3$ khối residual.

Cơ chế kết nối tắt (skip connection) trong mỗi khối residual cho phép gradient lan truyền hiệu quả, giúp mạng hội tụ ổn định ngay cả sau khi cắt tỉa lớn. ResNet-20 đóng vai trò kiến trúc tham chiếu chính (primary baseline) và được sử dụng trong tất cả các thử nghiệm so sánh do chi phí tính toán thấp, phù hợp với việc chạy nhiều cấu hình thử nghiệm.

4.2.2 ResNet-50

ResNet-50 [4] là phiên bản sâu hơn với 50 lớp, sử dụng kiến trúc khối *bottleneck* (conv 1×1 – conv 3×3 – conv 1×1) với khoảng **25,6 triệu tham số**. Mạng gồm bốn stages với lần lượt 64, 128, 256 và 512 kênh đặc trưng (feature channels).

ResNet-50 được sử dụng trên CIFAR-100 để đánh giá liệu các *winning ticket* có tồn tại và duy trì hiệu năng cao trong mô hình có dung lượng lớn hay không. Khối lượng tham số cao hơn cũng giúp kiểm tra khả năng mở rộng của thuật toán Genetic Algorithm và Hybrid-Improve khi không gian tìm kiếm tăng lên.

4.3 Siêu tham số

4.4 Chỉ số đánh giá

Để so sánh toàn diện giữa các thuật toán pruning, luận văn sử dụng ba nhóm chỉ số đánh giá: hiệu năng dự đoán, mức độ nén mô hình, và hiệu quả tính toán.

Table 3: Siêu tham số đặc thù cho từng thuật toán pruning (CIFAR-10/ResNet-20).

Thuật toán	Siêu tham số	Giá trị	Ý nghĩa
IMP	target_sparsity	0.3, 0.6	Tỷ lệ tham số bị cắt tỉa
	num_iterations	5	Số vòng lặp pruning
	epochs_per_iteration	100	Số epoch mỗi vòng lặp
	use_global_pruning	True	Pruning toàn cục hay theo lớp
	learning_rate	0.1	Tốc độ học
	batch_size	128	Kích thước batch
	weight_decay	0.0005	Hệ số regularization
Early-Bird	target_sparsity	0.3, 0.6	Tỷ lệ kênh bị cắt tỉa
	search_epochs	17–19	Số epoch tìm kiếm vé (thực tế)
	finetune_epochs	20–100+	Số epoch tinh chỉnh (thực tế)
	l1_coef	10^{-4}	Hệ số phạt L1 trên γ BN
	distance_threshold	0.1	Ngưỡng hội tụ mặt nạ
GraSP	target_sparsity	0.3, 0.6	Tỷ lệ tham số bị cắt tỉa
	epochs	160	Số epoch huấn luyện sau pruning
	samples_per_class	10	Số mẫu/lớp để tính GraSP score
	grasp_T	200.0	Nhiệt độ scaling logits
	grasp_iters	1	Số lần tích lũy gradient
	learning_rate	0.1	Tốc độ học
	batch_size	128	Kích thước batch
	weight_decay	0.0001	Hệ số regularization
SynFlow	rho	1.43, 2.5	Tỷ lệ nén ρ (tương ứng sparsity 0.3, 0.6)
	target_sparsity	0.3, 0.6	Tỷ lệ tham số bị cắt tỉa (thực tế)
	synflow_iters	100	Số vòng lặp SynFlow
	epochs	160	Số epoch huấn luyện sau pruning
	learning_rate	0.1	Tốc độ học
	batch_size	128	Kích thước batch
	weight_decay	0.0001	Hệ số regularization
Hybrid	target_sparsity	0.3, 0.6	Tỷ lệ cắt tỉa mục tiêu
	oneshot_ratio	0.7	Phần one-shot của mục tiêu
	iterative_step	0.1	Phần trọng số cắt mỗi bước lặp
	initial_epochs	160	Epoch huấn luyện dày đặc
	initial_lr	0.01	LR ban đầu pha huấn luyện dày đặc
	oneshot_finetune_max_epochs	200	Epoch tinh chỉnh sau one-shot
	oneshot_finetune_patience	200	Patience tinh chỉnh one-shot
	iter_finetune_max_epochs	10	Epoch tinh chỉnh mỗi bước lặp
	iter_finetune_patience	10	Early stopping bước lặp
	batch_size	128	Kích thước batch
	weight_decay	0.0005	Hệ số regularization
Hybrid-Improve	target_sparsity	0.3, 0.6	Tỷ lệ cắt tỉa mục tiêu
	oneshot_ratio	0.79	Phần one-shot thích ứng
	iterative_step	0.2	Phần trọng số cắt mỗi bước lặp
	initial_epochs	160	Epoch huấn luyện dày đặc
	initial_lr	0.1	LR ban đầu pha huấn luyện dày đặc
	oneshot_finetune_max_epochs	200	Epoch tinh chỉnh sau one-shot
	oneshot_finetune_patience	200	Patience tinh chỉnh one-shot
	iter_finetune_max_epochs	10	Epoch tinh chỉnh mỗi bước lặp
	iter_finetune_patience	10	Early stopping bước lặp
	batch_size	512	Kích thước batch
	weight_decay	0.0005	Hệ số regularization
	use_fisher_adaptive_ratio	True	Sử dụng Fisher adaptive ratio
	use_kd	True	Sử dụng knowledge distillation
	kd_temperature	4.0	Nhiệt độ KD
	kd_lambda	0.5	Hệ số KD

4.4.1 Độ Chính Xác Phân Loại

Top-1 Test Accuracy là chỉ số hiệu năng chính, đo tỷ lệ phần trăm mẫu kiểm tra được phân loại đúng:

$$\text{Acc} = \frac{\text{Số mẫu dự đoán đúng}}{\text{Tổng số mẫu kiểm tra}} \times 100\% \quad (4.1)$$

Để đảm bảo kết quả đáng tin cậy, mỗi cấu hình thử nghiệm được chạy với nhiều random seed và kết quả được báo cáo dưới dạng giá trị trung bình $\pm$ độ lệch chuẩn.

Accuracy Drop (ΔAcc) đo mức độ suy giảm độ chính xác so với mô hình gốc (dense network) tại cùng kiến trúc và bộ dữ liệu:

$$\Delta\text{Acc} = \text{Acc}_{\text{dense}} - \text{Acc}_{\text{pruned}} \quad (4.2)$$

Giá trị ΔAcc nhỏ cho thấy phương pháp pruning bảo toàn tốt hiệu năng gốc.

4.4.2 Chỉ Số Nén Mô Hình

Sparsity (s) là tỷ lệ trọng số bị cắt tĩa (đặt về 0) trên tổng số trọng số của mô hình:

$$s = \frac{|\{w_i : w_i = 0\}|}{|\{w_i\}|} \times 100\% \quad (4.3)$$

Sparsity 90% nghĩa là chỉ còn 10% trọng số không bằng 0.

Compression Ratio (ρ) biểu diễn mức độ nén theo tỷ lệ:

$$\rho = \frac{1}{1 - s} \quad (4.4)$$

Ví dụ, sparsity 90% tương đương $\rho = 10\times$ (mô hình được nén 10 lần).

Parameters Remaining (P_r) là số lượng tuyệt đối hoặc tỷ lệ trọng số còn lại sau pruning:

$$P_r = (1 - s) \times P_{\text{total}} \quad (4.5)$$

Chapter 5

Kết quả

Chapter 6

Thảo luận

Chapter 7

Kết luận

Bibliography

- [1] Eirik Fladmark, Muhammad Hamza Sajjad, and Laura Brinkholm Justesen. Exploring the performance of pruning methods in neural networks: An empirical study of the lottery ticket hypothesis. 2023. arXiv:2303.15479.
- [2] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. 2019. arXiv:1803.03635.
- [3] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. 2019. arXiv:1803.03635.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [5] Mikołaj Janusz, Tomasz Wojnar, Yawei Li, Luca Benini, and Kamil Adamczewski. One shot vs. iterative: Rethinking pruning strategies for model compression. 2025. arXiv:2508.13836.
- [6] Mikołaj Janusz, Tomasz Wojnar, Yawei Li, Luca Benini, and Kamil Adamczewski. One shot vs. iterative: Rethinking pruning strategies for model compression. 2025. arXiv:2508.13836.
- [7] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [8] Yann LeCun. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1998.
- [9] Saehyung Lee, Se Jung Kwon, Byeongwook Kim, and Sungroh Yoon. Revisiting the lottery ticket hypothesis for pre-trained networks. <https://openreview.net/forum?id=9ZUz4M55Up>, 2024.
- [10] Hidenori Tanaka, Daniel Kunin, Daniel Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. 2020. arXiv:2006.05467.

- [11] Chaoqi Wang, Guodong Zhang, and Roger Grosse. Picking winning tickets before training by preserving gradient flow. 2020. arXiv:2002.07376.
- [12] Haoran You, Chaojian Li, Pengfei Xu, Yonggan Fu, Yue Wang, Richard G. Baraniuk, Yingyan Lin, Xiaohan Chen, and Zhangyang Wang. Drawing early-bird tickets: Towards more efficient training of deep networks. In *International Conference on Learning Representations (ICLR)*, 2020. Preprint / ICLR 2020.
- [13] Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. 2019. arXiv:1905.01067.